

컴퓨터비전 개론 2주차 요약본

6. 손실함수 정의(Loss Function)

6.1 손실함수 (Loss Function)

- 비용 함수(Cost Function) : 전체 데이터(또는 배치)에 대한 평균 오차를 계산하는 함수
- 손실 함수란(Loss Function) : 모델이 예측한 값과 실제 정답 간의 차이를 수치로 표현한 함수

6.2 손실 함수의 종류

- 회귀 예측(Regression)
 - 지도학습(Supervised Learning)의 한 종류
 - 목표값(타겟, Target)이 연속적인 값을 가지는 경우 사용
 - MSE(Mean Squared Error, 평균 제곱 오차) 사용
 - 예: 집값 예측, 수율 예측, 혈압 예측 등
- 이진 분류(Binary Classification)
 - 지도학습 방식
 - 목표값이 두 개의 클래스로 나뉘는 경우
 - 활성화 함수로 Sigmoid(시그모이드) 사용
 - 출력값을 0과 1 사이로 변환하여 확률값으로 해석 가능
 - Binary Crossentropy 사용
 - 예: 종양(악성/양성) 판별, 스팸 메일, 시험 결과(P/F) 등
- 다중 분류(Multinomial Classification)
 - 목표값이 3개 이상의 클래스로 나뉘는 경우
 - 활성화 함수로 Softmax 사용
 - 각 클래스에 대한 확률값을 계산하여 총합이 1이 되도록 정규화

- Crossentropy (Categorical Crossentropy) 사용
- 예: 붓꽃의 종류(Setosa / Versicolor / Virginica), 성적 등급 산출(A~F) 등

6.3 클래스 불균형

- 분류 문제에서 각 클래스의 데이터 개수가 크게 차이 나는 상황을 의미함.
 - 예: 스팸 메일 5%, 정상 메일 95%

구분	방법	적용 단계	핵심 아이디어
가중치 손실 (Weighted Cross Entropy)	Weighted CE	모델 학습 시	클래스별 손실 가중치를 다르게 부여하여 소수 클래스에 더 큰 비중을 줌
과샘플링 (Over-sampling)	Random Over-sampling	데이터 전처리	소수 클래스 데이터를 복제하거나 늘려 데이터 비율을 맞춤
언더샘플링 (Under-sampling)	Random Under-sampling	데이터 전처리	다수 클래스 데이터를 줄여 클래스 비율을 맞춤
SMOTE	Synthetic Minority Over-sampling Technique	데이터 전처리	소수 클래스 데이터를 기존 샘플을 바탕으로 합성하여 생성
포컬 로스 (Focal Loss)	Focal Loss	모델 학습 시	어려운 샘플에 더 집중하도록 손실 함수를 조정

7. 선형 회귀(Linear Regression)

7.1 선형 회귀

- 개념
 - 지도학습(Supervised Learning)의 한 종류
 - 목표값(반응변수, Dependent Variable)이 연속형인 경우 사용
 - 입력 변수(Feature)가 하나일 경우 단순 회귀(Simple Regression)
 - 입력 변수가 여러 개일 경우 다중 회귀(Multiple Regression)
- 선형 회귀 모델

$$y = \beta_0 + \beta_1 x + \epsilon$$

- y: 종속 변수(목표값)
- x: 독립 변수(입력값)
- β_0 : 절편(intercept)
- β_1 : 기울기(slope)
- ϵ (epsilon): 오차(error term)

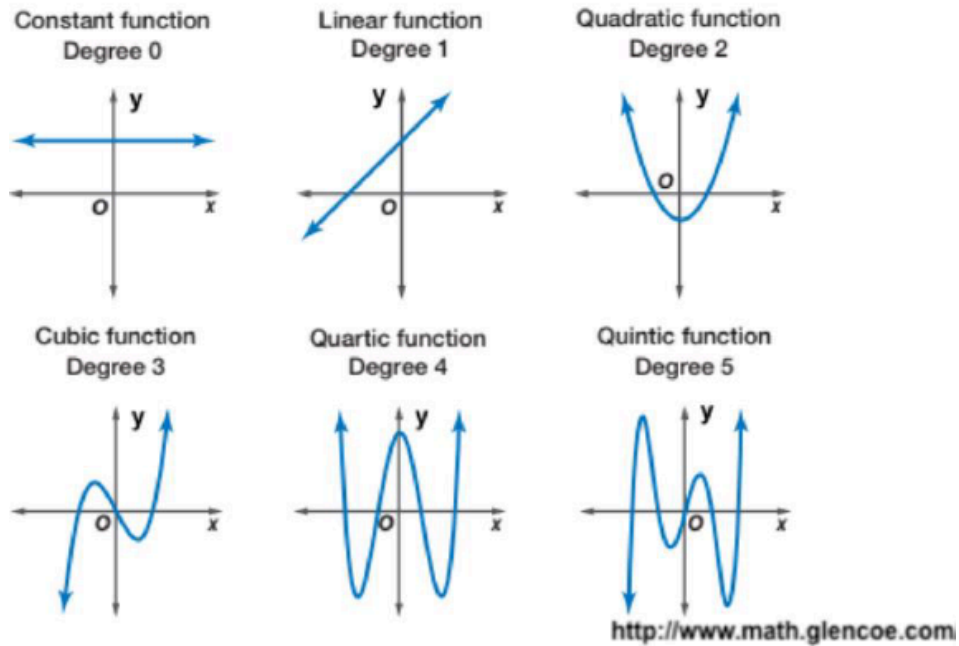
7.2 최소제곱추정량 (OLS, Ordinary Least Squares Estimation)

- 최소제곱법의 개념
 - 회귀선이 데이터를 가장 잘 표현하도록 가중치를 찾는 방법
 - 오차(Residual, 실제값 - 예측값)의 제곱합을 최소화하는 방식
 - 오차의 제곱을 최소화하는 방식으로 최적의 회귀선을 찾음
 - 경사 하강법(Gradient Descent) 또는 행렬 연산을 이용하여 최적화 가능

7.3 다항 회귀(Polynomial Regression)

- 다항 회귀란?
 - 독립 변수의 차수가 2차 이상인 회귀 모델
 - 비선형 관계(Non-Linear Relationship)를 모델링 할 때 사용
 - 단순 선형 회귀와 달리 비선형 관계를 설명 가능
 - 변수의 차수를 증가시키면, 보다 복잡한 패턴을 모델링 할 수 있음

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_p x^p + \epsilon_i$$

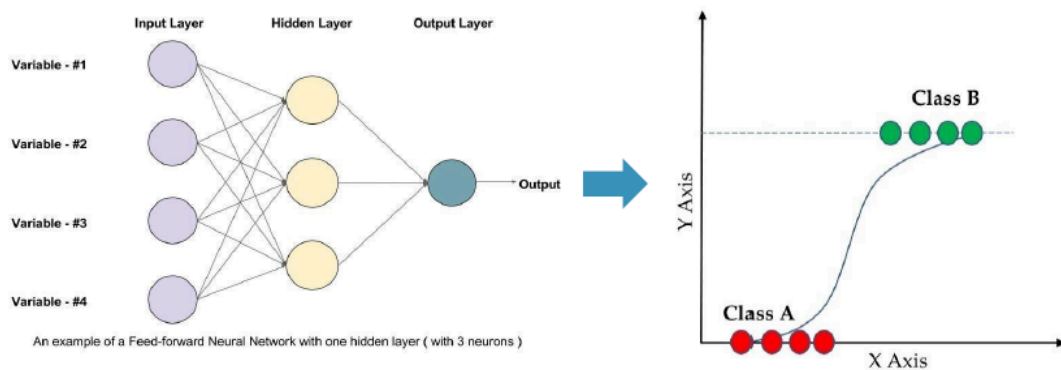


8. 이진 분류(Binary Classification)

8.1 이진 분류 모델 구조

• 기본 구조

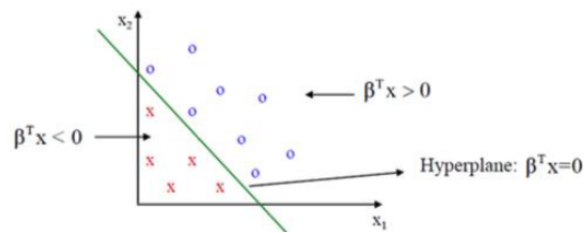
- 입력층 (Input Layer) → 가중치(W) 및 활성화 함수(Activation Function) → 출력층 (Output Layer)
- 출력층에서는 확률 값을 기반으로 이진 분류 수행



• 결정 경계 (Hyperplane)

- 데이터를 두 개의 클래스로 구분하는 경계를 의미
- 선형 모델에서는 직선(2D) 또는 평면(3D)으로 결정 경계를 형성

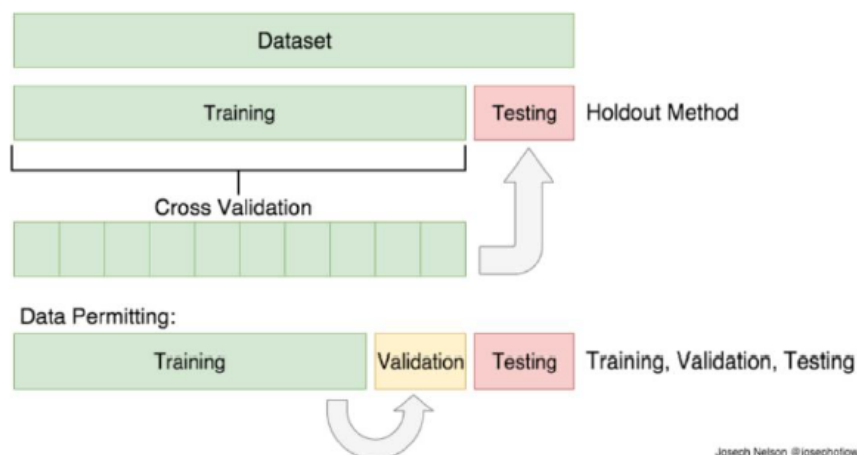
결정 경계 구하기 (Hyperplane)



8.2 훈련 데이터셋과 검증&평가 데이터셋

• 데이터셋 분할

- 학습 데이터 (Training Set): 모델을 학습하는 데 사용
- 검증 데이터 (Validation Set): 하이퍼파라미터(Hyper Parameter) 최적화를 위한 데이터
- 테스트 데이터 (Testing Set): 모델 성능을 평가하는 데 사용



8.3 모델 평가 방법

• 교차 검증 (Cross Validation)

- 데이터셋을 여러 개의 부분으로 나누어 여러 번 학습 및 평가를 수행
- k-fold 교차 검증이 대표적인 방법

• 부트스트래핑 (Bootstrapping)

- 데이터의 일부를 샘플링하여 여러 개의 학습 데이터셋을 생성하는 방법
- 데이터가 적을 때 유용

8.4 모델 평가 지표

1. 정확도 (Accuracy)

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

전체 데이터 중에서 올바르게 분류된 샘플의 비율.

2. 정밀도 (Precision)

$$Precision = \frac{TP}{TP + FP}$$

모델이 양성(Positive)으로 예측한 것 중 실제 양성인 비율.

예제: 스팸 필터 → 거짓 양성(False Positive)이 중요한 경우.

3. 재현율 (Recall, Sensitivity)

$$Recall = \frac{TP}{TP + FN}$$

실제 양성(Positive) 데이터 중 모델이 올바르게 예측한 비율.

예제: 암 진단 → 거짓 음성(False Negative)이 중요한 경우.

4. F1-점수 (F1-Score)

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

정밀도(Precision)와 재현율(Recall)의 조화 평균.

8.5 혼동 행렬 (Confusion Matrix)

	Positive Predicted (P)	Negative Predicted (N)
Positive Actual(P)	True Positive (TP)	False Negative (FN)
Negative Actual(N)	False Positive (FP)	True Negative (TN)

- **True Positive (TP):** 실제 양성을 양성으로 예측
- **False Positive (FP):** 실제 음성을 양성으로 예측 (거짓 양성)
- **False Negative (FN):** 실제 양성을 음성으로 예측 (거짓 음성)
- **True Negative (TN):** 실제 음성을 음성으로 예측

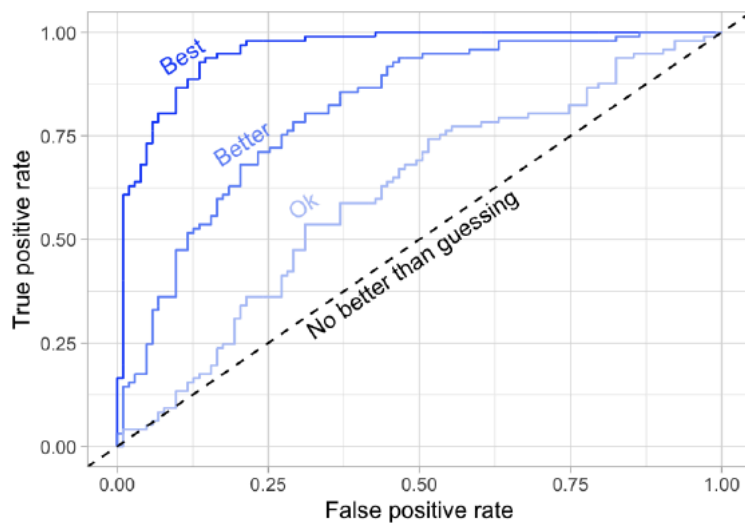
8.6 평가 지표 선택 기준

평가 지표	중점 사항	예시
정밀도(Precision)	거짓 양성(FP)을 줄이는 것이 중요할 때	스팸 필터, 광고 클릭 예측
재현율(Recall)	거짓 음성(FN)을 줄이는 것이 중요할 때	암 진단, 사기 탐지
F-1 Score	정밀도와 재현율이 모두 중요한 경우	일반적인 이진 분류 문제
ROC-AUC	확률 기반 모델의 전체적인 성능 평가	다양한 확률 분류 문제

8.7 ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

- 확률 기반 모델의 전반적인 분류 능력을 평가하는 지표
- AUC 값이 1에 가까울수록 모델 성능이 좋음

AUC



8.8 Pytorch 학습 과정의 핵심 단계

1. 경사값 초기화

`optimizer.zero_grad()` # 이전 단계의 gradient 초기화

2. 순전파

`outputs = model(inputs)` # 모델에 입력 → 예측값 계산

3. 손실 계산

loss = criterion(outputs, labels) # 예측값과 실제값 비교

4. 역전파

loss.backward() # 각 파라미터의 gradient 계산

5. 파라미터 업데이트

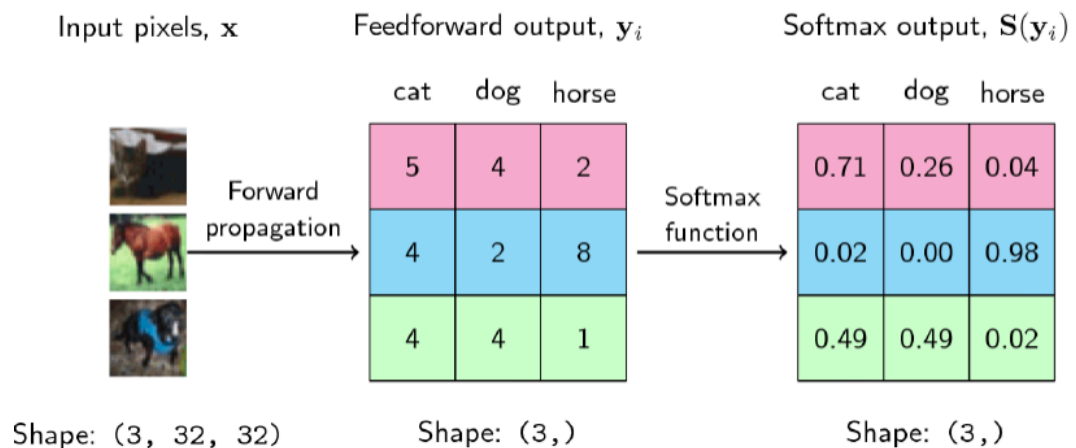
optimizer.step() # 경사를 이용해 파라미터 업데이트

9. 다중 분류(Multinomial Classification) 1

9.1 다중 분류 모델 구조

- 딥러닝에서 다중 분류 모델의 기본 구조

- 입력층 (Input Layer) → 가중치(W) 및 활성화 함수(Softmax Function) → 출력층 (Output Layer)
- 출력층에서 Softmax 함수를 사용하여 각 클래스에 대한 확률을 계산



- Softmax 함수의 역할

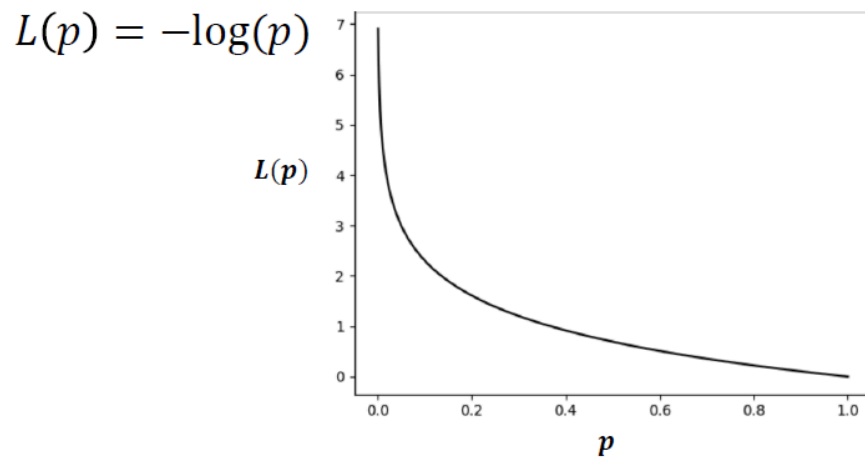
- 다중 클래스 분류에서 각 클래스에 대한 확률을 제공
- 확률 값이 가장 높은 클래스를 최종 예측값으로 선택

9.2 Negative Log-Likelihood (NLL)

- NLL 개념

- 모델의 출력 확률과 실제 클래스 간의 차이를 측정하는 손실 함수

- 예측이 정확할수록 NLL 값이 작아짐



p : 모델이 예측한 해당 클래스의 확률값 NLL 값이 작을수록 모델의 예측 성능이 좋음

9.3 다중 분류에서 Negative Log-Likelihood (Multiclass NLL)

$$J(\theta) = - \sum_{i=1}^n y_i \log h_{\theta}(x_i)$$

- y_i : 실제 클래스 레이블 (One-hot Encoding 사용).
- $h_{\theta}(x_i)$: Softmax를 적용한 모델의 예측 확률값.

다중 클래스 분류에서 손실 함수로 사용 → 모델이 특정 클래스를 예측할 확률이 높을수록 손실값이 작아짐.

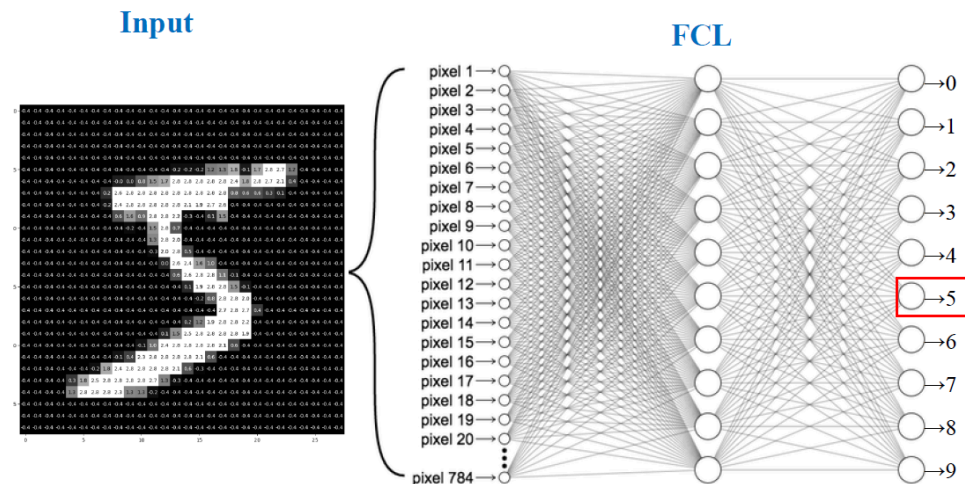
10. 다중 분류 2

10.1 MNIST dataset에 대한 이해

- 이미지 인식, 분류 모델 학습 및 테스트용 데이터셋
- 손글씨로 작성된 0~9 숫자(10종)으로 구성
- 총 7만개의 데이터 (훈련용 이미지 6만개, 테스트용 이미지 1만개)
- 28×28 픽셀의 흑백 이미지

10.2 Fully connected layers를 이용한 MNIST dataset 분류

- Fully connected layers(FCL)
 - MLP(다층 퍼셉트론)라고도 함
 - CNN없이 학습 가능



- 입력: $28 \times 28 = 784$ 차원
- 출력: 10 클래스(0~9 숫자)
- 예제 코드
 - Torchvision dataset 모듈 사용
 - Torchvision transforms 를 활용한 이미지 데이터 전처리
 - Pytorch Dataloader를 이용한 Mini-batch 생성

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

# 1. 데이터셋
# Torchvision transforms를 활용한 이미지 데이터 전처리
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
```

```

])

train_data = datasets.MNIST(root="./data", train=True, download=True,
transform=transform)
test_data = datasets.MNIST(root="./data", train=False, download=True,
transform=transform)

# Pytorch DataLoader를 이용한 Mini-batch 생성
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)

# 2. 모델
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(28*28, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 28*28)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

model = MLP()

# 3. 손실 함수 & 옵티마이저
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 4. 학습 루프
epochs = 3
for epoch in range(epochs):
    model.train()
    total_loss = 0
    for images, labels in train_loader:
        optimizer.zero_grad()

```

```

        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {total_loss/len(train_loader):.4f}")

# 5. 평가
model.eval()
correct = 0
total = 0
all_preds = []
all_images = []
with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
        # 일부 이미지와 예측 저장
        all_images.extend(images[:8]) # 배치에서 앞 8개만 저장
        all_preds.extend(predicted[:8])
        break # 한 배치만 시각화에 사용
print(f"테스트 정확도: {100 * correct / total:.2f}%")

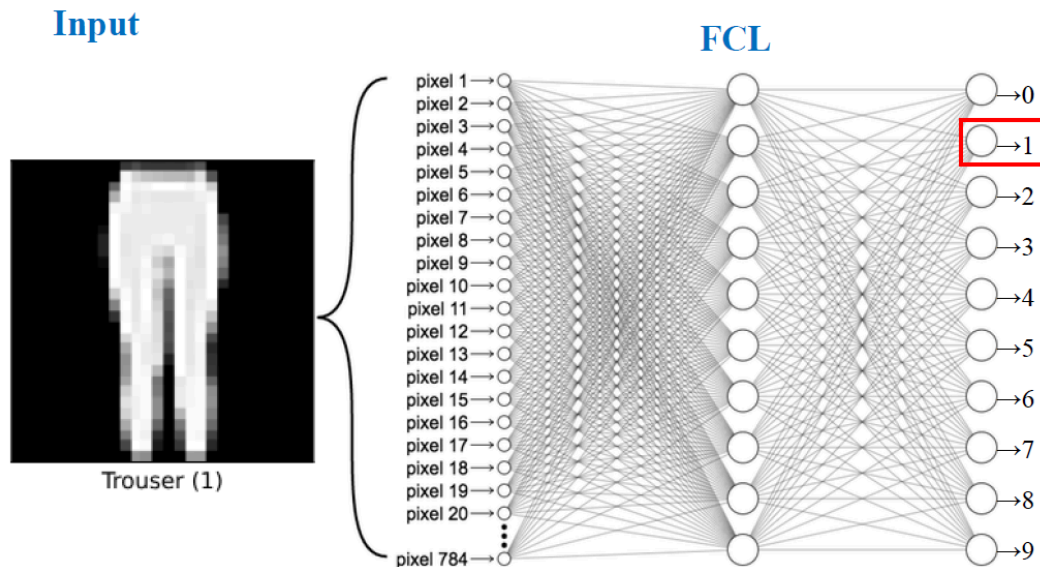
# 6. 결과 이미지 시각화
fig, axes = plt.subplots(2, 4, figsize=(10, 5))
for i, ax in enumerate(axes.flat):
    img = all_images[i].squeeze().numpy() # (28×28)
    ax.imshow(img, cmap="gray")
    ax.set_title(f"Pred: {all_preds[i].item()}")
    ax.axis("off")
plt.show()

```

10.3 Fully connected layers를 이용한 Fashion MNIST dataset 분류

- **Fashion-MNIST:** 의류 이미지(티셔츠, 바지, 신발 등) 28×28 픽셀

- MNIST와 구조는 같지만 난이도가 조금 더 높음
- 같은 네트워크 구조를 그대로 적용 가능



10.4 딥러닝 신경망의 구조

- **입력층 (Input Layer)**
 - 데이터가 모델로 처음 들어오는 층
 - 각 노드(뉴런)가 입력 데이터의 한 특징(feature)을 나타냄
예: 이미지(28×28) → 입력 노드 784개
- **은닉층 (Hidden Layer)**
 - 입력과 출력을 연결하며 패턴을 학습하는 층
 - 여러 개의 가중치(Weight)와 활성화 함수(Activation Function)로 구성
 - 비선형 관계를 학습하는 핵심 부분
예: ReLU, Sigmoid, Tanh 등 사용
- **출력층 (Output Layer)**
 - 모델의 최종 결과를 내는 층
 - 회귀 → 1개의 노드 (예: 예측값)
 - 분류 → 클래스 수만큼 노드 (예: Softmax로 확률 출력)

10.4 Pytorch 데이터 전처리

구성요소	역할	예시
Dataset	데이터셋을 정의 (입력, 라벨 관리)	<code>torch.utils.data.Dataset</code>
Transforms	이미지/데이터 전처리, 변환	<code>torchvision.transforms</code>
DataLoader	배치 단위로 데이터 불러오기, 섞기, 병렬처리	<code>torch.utils.data.DataLoader</code>

10.5 배치 사이즈

구분	작은 배치	큰 배치
메모리 사용량	적음	많음
학습 속도	느림	빠름
기울기 안정성	불안정 (진동 가능)	안정적
일반화 성능	좋음 (과적합 ↓)	떨어질 수 있음 (과적합 ↑)
과적합 위험	낮음	높음
추천 상황	일반화 중시, 메모리 적을 때	대용량 데이터, 학습 속도 중요할 때

10.6 클래스 불균형

- 한 클래스의 데이터가 다른 클래스보다 **극도로 적거나 많은 현상**
 - 예: 스팸메일(1%) vs 일반메일(99%)
- 모델이 **다수 클래스에만 치우쳐 학습** → 소수 클래스 예측 성능 저하
- Accuracy가 높아도 실제 성능이 낮을 수 있음 (예: 모두 정상으로 예측)
- 클래스 불균형 해결 방법

방법	개념	특징 / 장점
가중 손실 (Weighted Loss)	손실 함수에 클래스별 가중치 부여 → 소수 클래스의 오차를 더 크게 반영	데이터 비율에 따라 자동 조정 가능모델 구조 변경 없음
SMOTE (Synthetic Minority Over-sampling Technique)	소수 클래스 데이터를 인공적으로 생성(보간) 하여 샘플 수 증가	단순 복제보다 다양성 확보, 과적합 완화
데이터 증강 (Data Augmentation)	이미지 회전·뒤집기 등으로 소수 클래스 데이터 다양화	특히 이미지/음성 데이터에 효과적
앙상블 기법 (Ensemble Methods)	여러 모델을 결합하여 소수 클래스 탐지 강화 (예: Bagging, Boosting)	불균형 데이터에서도 안정적 성능 확보